

01 BU PROBLEM NEDEN VAR

Tanım Otonom bir sistemin karar ver → araç çağır → gözlemler → gözden geçir → tekrarla döngüsünü sürdürmesi, ama bir sonuca varmaması. Modexa'nın ifadesiyle: "sistemlerin ilerleme kaydetmeyi bıraktıktan çok sonra bile düşünmeye, araç çağırılmaya, tekrar denemeye veya yeniden planlamaya devam etmesi."		Beş gerçek sebep — Modexa Çoğu döngü "kötü modellerden" kaynaklanmaz. Kötü teşvikler ve eksik kısıtlamalardan kaynaklanır.		ÖLÇÜLMÜŞ KANIT — TEORİK DEĞİL	
Neden bu soru modelde yoktu: bir dil modeli tek başına durur — sorarsınız, cevap verir, biter. Ajan = model + döngü. Ajanın bütün gücü buradan gelir, ama modelde hiç olmayan bir soru da buradan doğar: bu döngüyü kim kesecek?		1 Açık bir "bitti" tanımının olmaması. İnsanların içsel bir "bu kadar yeterli" algısı vardır; siz vermedikçe ajanların yoktur. Prompt "kapsamlı ol: doğrula: hiçbir şeyi kaçırma" dıyorsa, ajana sonsuza kadar arama yapmasını söylemişsinizdir. <i>"Ajan bozuk değildir. Sadece verdiğiniz emirlere itaat ediyordur."</i>		MAST · 1642 iz, 7 framework — adım tekrarı %15,7 (en sık) + durma koşulunu tanımama %12,4	
En doğal cevap "model kendisi karar versin"; pratikte en kötü cevap bu. Sebep modelin zekâsı değil görüş alanı: her turda önündeki konuşmaya bakıp bir sonraki adımı seçiyor, kaç kez aynı şeyi denediğini ve ne harcadığını görmüyor.		2 Güvenilmez araçlar + naif retry. Araç tutarsızsa (zaman aşımı, hız sınırı, kısmi yanıt) ajan bunu "tekrar dene, ama biraz farklı" diye yorumlar. Üst sınır yoksa sistem bir slot makinesine dönüşür.		Token Budgets · 21 framework — "katalogda bir kullanıcı parasını ödmeden önce engellenmiş tek bir vaka yok"	
"Ajanlar, insanların döngüye girmesiyle aynı nedenden dolayı döngüye girerler: ilerleme kaydetmediklerinin farkında değildirler."		3 Belirsiz hedef. "Şunu düzelt" bir gereksinim değildir. Başarı kriteri belirsizse ajan yorumlar arasında gidip gelir — "saatlerce aynı paragrafı yeniden yazan bir öğrenci gibi". Hedef belirsizliği, sistem üzerinde döngü baskısı yaratır.		Anthropic referans döngüsü — while True, tur sayacı yok, döngü tespiti yok, bütçe yok	
Dışarıdan nasıl görünür: sonsuz tekrarlar ("belki araç başarısız oldu?") · bitmeyen aramalar ("bir kaynak daha bulayım") · özleştirir sarmalları ("cevabım yanlış olabilir") · araç ping-pong'u (A→B→A→B) · plan çalkantısı (her adımda planı baştan yaz). Ajan sürekli hareket halindeydi ama sistem ilerlemiyordu. Hareket etmek, ilerlemek demek değildir.		4 Bağlam her adımda kötüleşir. Her adım yeni token ekler: loglar, kısmi planlar, araç çıktıları. Sonunda ajan kendi kafa karışıklığının dağınık bir transkripti üzerinden mantık yürütmeye başlar — ve kafa karışıklığı daha fazlasını besler.		31 harness kaynak kodundan — mekanizma varsayımları var ama varsayımları kapalı · ADK LoopAgent: None=sınırsız	
		5 Ajanın "yanlış yapmak" üzerine optimize edilmesi. Aşırı tedbir teşvik eden bir prompt'ta en güvenli yol "bir kontrol daha yap" olur. "Bir sorun çözücü inşa etmediniz. Bir riskten kaçınma makinesi inşa ettiniz." Bu sebep başka hiçbir kaynakta yok — ve kaynağı kod değil prompt, yani bu sayfadaki hiçbir mekanizma onu çözmiyor.		A-B-A-B döngü — ardışık tekrar sayarın sayacı her adımda sıfırlanır	
		Zihinsel model Ajanları arama algoritması gibi düşünün. Şunlardan yoksun bir arama başıboş dolaşır: durma koşulu · bütçe · ilerleme sezgiseli. Yani döngüler eksik kısıtlamalardır. Çözüm "ajana durmasını söylemek" değildir — sisteme durması için MEKANİK NEDENLER vermektir.		Gerçek LLM, sağlıklı senaryo (kendi ölçümüm) — hiçbir şey bozuk değilken 40 çağrının 40'ı aynı tıklama	
				Doğru soru "Bu framework'te loop detection var mı" değil — "varsayımları açık mı". Hermes'te aynı fonksiyonun gerçek varsayımları sys.maxsize, docstring'i "default: 90" .	
				"Ajanlar sadece yanlışlıkları için başarısız olmazlar. İsrarcı oldukları için başarısız olurlar. Ve ısrar pahalıdır." "Durma koşulları olmayan otonomi, sadece bir yangındır."	

02 ÇÖZÜM KATEGORİLERİ — BEŞİ DE AYNI SORUYA FARKLI CEVAP

sıra = uygulama önceliği · sayılar: dead_button · senaryosu · kontrolsüz = 300 adım / 105,000 token / \$1,050

ZİHNİYET	NASIL DAVRANIR — KAYNAĞIN AYIRT EDİCİ FİKRİ	TETİKLENDİĞİNDE NE YAPAR	DURUM ADIM	\$ NEYİ KAÇIRIR
SAYAC — ne yaptığını anlamaya çalışma, kaç yaptığını say · bir tamsayı, durum yok, O(1) · dördü de AYNI sayacı kullanıyor, fark sayac dolduğunda ne olduğu				
1 arize-control Arize — Agent Control Loop	Beş eksenle sayar: adım · replan · token · süre · dolar. Aralarında hiyerarşi var — adım limiti birincil, çünkü diğer sinyaller yorum gerektirir ("görev tamamlandı" modelin görüşü, "hata" tanıma bağı) ama on iki adım attıysa on iki adım atmıştır.	SERT keser — lütf turu yok, cevap payı yok. İkinci katkısı: durma sebebi kaydeder. "%70 başarı" diyen bir sistemde kalan %30'un ne olduğu her şeyi değiştirir: hepsi limitte bitiyorsa limit dar, hepsi hata veriyorsa araçlar bozuk.	BUDGET_EXHAUSTED	13 0,042 Ne olduğunu söylemez — sadece "çok oldu". Kandırılmaz ama teşhis değil, tavandır. Pencere dedektörüyle birlikte şart.
2 budget-grace AgentScope + Hermes 2 varyant	Aynı sayac. Gözlemleri şu: sert tavan ajani için ortasında keser ve o ana kadarki iş çöpe gider — sonucu toparlayacak bütçesi kalmamıştır. Ama "lütfe bitir" demek de yetmez, model dinlemeyebilir.	Limit dolunca 1-5 ek tur verir ve araç seçimini kilitler: ajan yeni araç çağırılmaz, yalnız cevap üretilebilir. Nazik rica değil mekanik kısıt. Zorla biten koşum OK dönmez — tüketiş görünür kalır.	DEGRADED	13 0,042 Lütf turları da para yakar: İki varyant farkı: AgentScope 5 tur/uyarı yok · Hermes 1 tur/yalnız süre ekseninde uyarı.
3 claudes-advistory Claude Task Budgets	Tam tersini savunur: dışarıdan kesmek kaba. Geri sayımı prompt'a enjekte eder, model kalan bütçesini görür ve işi kendi toparlar. Hermes ile karşıtlık: Hermes bunu denedi ve kaldırdı — "ora basınc uyarıları modelleri karmaşık görevlerde erken pes ettiriyordu."	Tavsiye, zorlama değil — model, kesilmesi bitirilmesinden zararlıysa bütçeyi aşabilir. Amaç durdurmak değil öngörülebilir inış. Bizim aldığımız ders: eksene göre karar ver — adımda uyarma, sürede uyar.	BUDGET_EXHAUSTED	13 0,042 Zorlayıcı değil; model dinlemezse hiçbir koruma yok. Ve bütçe yetersizse model işi hiç denemiyor — "her ihtimale karşı düşük tutayım" geri tepiyor.
4 agentbudget-dollar AgentBudget	Token bir vekil ölçü: iki model arasında 15x fiyat farkı olabilir, aynı token sayısı çok farklı fatura üretir. Sistemin dışındaki gerçek birimini sayar. Ayrıca tekrar tespitini zamana bağlar: "on çağrı" değil "bir dakikada on çağrı" — yavaş ama kronik döngü sayı eşliğine hiç ulaşmadan saatlerce yavaş.	Nihai cevap payı %15: sert limit erken tetiklenir, ajan tavana çatırtığında elinde hâlâ "bulduklarımı özetleyeyim" diyecek bütçe kalır. terminated_by ile hangi eksenin bitirdiği kaydedilir.	BUDGET_EXHAUSTED	12 0,038 Fiyat tablosu bakım ister; model değişince yanlış sayar. Rezerv oranı da tahmin — nihai cevabın gerçek maliyetinden ölçülmeli.
PENCERE — son N olayı tut, karşılaştı · sayacın söyleyemediğini söyler: NE yanlış gidiyor · bedeli tek soru: "aynı şey" ne demek?				
5 openhands-stuck OpenHands stuck_detector.py	Ne yapar: her eylemin bir parmak izini çıkarır, son 20 olayda beş ayrı desen arar — her birine ayrı eşik: aynı eylem+gözlem 4 · aynı eylem+hata 3 · monolog 3 · A-B-A-B 6 · context-window. Nasıl anlar: imzayı kurarken her turda zaten değişen alanları atar. Atmazsan iki özdes çağrı asla eşit çıkmaz — dedektör çalışır, log basar, hiçbir zaman hiçbir şey bulmaz. imza = sha256(kind + name + payload) ATILAN: tool_call_id, timestamp, request_id, trace_id, seed, nonce pencere = son 20 olay (son USER mesajından)	Kademelendirme yok — doğrudan durur. "Sıkıtıysan sıkışmışsındır." Ve STUCK ayrı bir terminal durum: ne başarı ne hata, üçüncü bir sonuç.	STUCK	3 0,011 İmza normalizasyonu yanlışsa dedektör çalışır, log basar, hiçbir zaman hiçbir şey bulmaz — ve testleri geçer. Hata türlerinin en kötüsü.
6 openclaw-pingpong OpenClaw 2026.7.1	Üretimde yılanmış hâli. Dedektörlerin adı var: genericRepeat / knownPollNoProgress / pingPong — hangi dedektörün konuştuğunu bilmek, ne olduğunu bilmek demek. Parmak izi = araç + argüman + sonuç hash'i.	Üç kademe 10 → 20 → 30 (uyarı → kritik → küresel devre kesici). Ve bir olgunluk göstergesi: sıkıştırma sonrası ayrı koruma — bağlam sıkıştırmanın kendisi döngü üretiyor, üç deneme boyunca özel gözetim.	STUCK	11 0,038 Kaynakta enabled varsayımları false. Üçüncü kademeye kadar çok para yonar — bizim ölçümde 11 adım, openhands 3.
7 pi-signature pi-anti-doom-loop	Kapattığı boşluk: ajan aynı şeyi farklı kelimelerle söylüyor — ls -la / ls -al / ls -a -l. Parmak izleri farklı, imza dedektörü görmez. Nasıl anlar: iki mesajın ortak kelime oranına bakar — küme keşimini. Gömme modeli yok: deterministik, anında, bedava. Yanına 5 sinyal daha, ayrı eşiklerle. A, B = set(m1.lower().split()), set(m2...) ortusme = len(A & B) / max(len(A), len(B)) if ortusme >= 0.55: aynı adimin yeniden ifadesi	Kademeli: metin döngüsünde önce yönlendir, sonra kes; araç çağrısında engelle → sebebini söyle → turu bitir. Engellenen çağrı pencereye girmez, ayrı bir blok sayacı tutulur.	STUCK	4 0,014 6 sinyal × ayrı eşik × iki müdahale merdiveni × sayac sıfırlama kuralları. Doğru uygulamak için hepsini akılda tutmak gerekiyor; yanlış uygulanırsa sessizce ölür.
8 strands-entropy Strands SDK	Tekrarın tanımından kaçır. Herkes tanım bataklığına giriyor: ardışık mı sayalım? A-B-A-B olursa? A-B-C-A-B-C olursa? Her desene ayrı tarama mı? Strands tersini sorar: "son N adımda kaç FARKLI şey oldu?" — len(set(pencere)) < eşik, tek satır.	Tek kural bütün desenleri yakalar; k=1..12 çevrim taraması gerekmez. Loop_budget_source bağımsız olarak aynı fikre varmış — iki ayrı kaynak, aynı içgörü.	STUCK	6 0,021 Mesrur olarak dar alanda çalışan iş de düşük çeşitlilik gösterir — bir dosyayı defalarca düzenleyip test eden ajan aslında ilerliyordur.
9 loopguard-dignity "Ne Zaman Durması Gerektiğini Bilen Ajanlar"	Kapattığı kör nokta: ajan her turda gerçekten farklı bir şey deniyor olabilir — hiçbir tekrar dedektörü tetiklenmez çünkü hiçbir şey tekrarlanmıyor, ama hiçbiride de işe yaramıyor. Nasıl anlar: eyleme değil dünyanın durumuna bakar — ekran hash'i 8 adım değişmediyse ilerleme yok. Ayrıca eylem başına retry sayar ve ekran değişince sıfırlar. son = ctx.screen_hashes[-8:] if len(son)==8 and len(set(son))==1: DUR retries[(eylem, argüman)] >= 3: DUR ekran degisti: retries.clear() # ilerleyen retry, # tüketen retry değil	Çekimser kalır — NEEDS_INPUT, hata değil ayrı bir sonuç. Ve dört alanlı rapor üretir: ne denedi - ne buldu - neden durdu - sırada ne var. "Bu, sonsuz döngüyü faydalı kısmı sonuca dönüştürür."	NEEDS_INPUT	7 0,025 Hızlı ve ucuz döngüler sınırlar dolmadan çok tur döner. Kaynağın eşikleri (3 ve 2) bu alana taşınmadı — ölçtük, mesrur koşumu başarıdan bir adım önce kesiyordu; 8'e çıkarıldı.

DUNYA — davranışı bırak, KANITA geç · ajanın en tehlikeli hatası döngüye girmek değil (o güvültülü, fatura kabarıp), yanlış işi doğru sanmak (o sessiz)				
10 verify-gate Loop Engineering	Ne yapar: model Finish dediğinde bu kabul edilmez — on_finish_claim kancası devreye girer. Nasıl anlar: kanıt ortamdan okur, modele sormaz. Gerçek sistemde bu testi koşturmak, dosyayı okumak, dış servise sormaktır. <pre>def on_finish_claim(fin, ctx): kanit = "gonderildi" in ctx.sandbox.describe() if kanit: return CONTINUE # kapi acildi return self._redDET(...) # NUDGE, sonra STOP</pre>	Kanıt yoksa NUDGE → koşum SÜRER , red sebebi ajanın gözlem akışına girer ("ekranda kanıt yok, iş bitmedi") ve ajan hatasını düzeltebilir. 2 redde ısrar ederse STOP. Gerçek iz: nudge → nudge → stop, 3 adım.	CEILING 300 1,050	Kör nokta kancanın adında: on_finish_claim — iddia gelmezse kanca HIÇ çağırılmaz . Gerçek LLM ile ölçüldü: 50 adım boyunca continue, en pahalı sütun. liar modeli → DEGRADED 3 adım; none aynı koşumu OK sayıp yapılmamış işi teslim ediyor.
11 telemetry-repair Real-Time Detection & Repair	Üç kural, sırayla; ilk düşenin ADI döner. Sıra dıştan içe — sonuç zaten yanlışsa yolu inceleme. Ne yapar: tek kapı değil üç ayrı kural , sırayla — ve ilk düşenin ADI döner. Sıra dıştan içe: sonuç zaten yanlışsa yolu incelemeyle gerek yok. Nasıl anlar: 1) ajanın iddiası ile dünyanın hâli çelişiyor mu 2) görevin gerektirdiği çağrılar izde var mı (küme kapsama) 3) araç boş çıktı mı döndürdü. İstatistik değil kural — eşik yok, model yok, kalibrasyon yok: ya sağlar ya sağlamaz. <pre># 1 total_consistency — SONUC: iddia <-> dünya if "gonderildi" not in ekran and "gonder" in iddia: dus("total_consistency") # 2 required_coverage — YOL: iz <-> görevin gereği yapılan = {e.name for e in events if e.kind is ACTION} if not {"type", "left_click"} <= yapılan: dus("required_coverage", eksik=gerekli-yapılan) # 3 tool_contract — GIRDİ: donen <-> sözleşme for e in events[-6:]: if e.kind is OBSERVATION and not e.payload["output"]: dus("tool_contract")</pre>	Checkpoint'e geri sarar ve düşen kuralın ADINI söyler. Ölçülen dört kademe: hiçbir şey deme %16 · genel uyarı %36 · doğru cevabı ver %36 · hangi kontrol düştü %45 . Cevabı verince ajan kopyalıyor, neden yanlışlığını anlamıyor.	CEILING 300 1,050	Checkpoint tutmak mimariye ciddi yük. Ve max_repairs=1 — ikinci ihalde onarım değil DURMA , çünkü aynı çalışma ölçmüs: döngüde yeniden çalıştırma aynı döngüyü üretiyor . liar → 2 adım
12 galileo-breaker Galileo AI	Ne yapar: araç başına kayan bir pencere tutar; o aracın son çağrılarının yüzde kaçının hata verdiğine bakar. Neden sayı değil oran: "üç hata" bir araç yüz kez çağırılıysa normaldir, dört kez çağırılıysa değildir. Pencere kaydığı için oran düşerse kesici açılmaz — flaky'de meşru retry bu yüzden kesilmiyor. <pre>calls[arac] = deque(maxlen=5) # arac BASINA calls[arac].append(hata_mi) # [ok,hata,hata,hata] if len(calls[arac]) >= 4: oran = sum(calls[arac])/len(calls[arac]) # .75 if oran >= 0.75: KESICI AC</pre>	İki kademe, bilecek ayrı. wasted_calls → kesmez, raporlar (asıl dert kesmek değil, " <i>hakkında hiç ticket açılmayan hatayı</i> " görünür kılmak). tool_circuit_open → oran eşigi aşarsa keser. Raporu ajani suçlamaz: " ajan doğru davrandı; zafiyet araç katmanında. "	CEILING 300 1,050	Bu senaryoda hiç hata yok — buton sessizce hiçbir şey yapıyor, bakacağı veri yok. Geçici ile kalıcı hatayı ayırması tamamen eşige bağlı. broken_tool → DEGRADED 5 adım (2. hatada uyardı, 3'te kesti)

ŞEKİL — tespiti bırak, döngünün OLUŞMASINA izin verme · yanlış pozitif problemi hiç doğmaz çünkü tespit yok · bedeli: mevcut ajana sonradan eklenemez				
13 modexa-statemachine Modexa	İzinli geçiş tablosu: ANLA→TOPLA·DEVRET TOPLA→EYLEM·TOPLA·DEVRET EYLEM→DOGRULA·DEVRET DOGRULA→YANIT·EYLEM·DEVRET DOGRULA→TOPLA yok — "doğrulamamı, baştan toplayayım" döngüsü imkânsız . Ve DOGRULA boş bir geçiş değil bir KAPI : eylem gerçekten bir şey değiştirdi mi?	Değiştirmediyse ajan kendi retry'ını icat etmez, merdivene girer — indeks yalnız ileri: backoff → araç değiştir → kapsam daralt → ku'llanıcıya sor → elindekiyle cevapla. Gerçek iz: 1→2→3. basamak, sonra çekimser. 4. basamağın sözleşmesi: tek soru · neyin değişeceğini açıkla · cevapsızlıkta varsayılan.	NEEDS_ 8 0,028 INPUT	Esnekliği azaltır; öngörülemez işler makineye sığmaz. Raporunun cümlesi zihniyetin özeti: " döngü TESPİT edilmedi — oluşmasına izin verilmedi. " Kaynağın beşinci döngü sebebini hiçbir mekanizma çözmiyor — kaynağı kod değil prompt.
KARAR — "ne zaman dur" yanlış soru · diğer 13'u döngünün KENARINDA durup izliyor, bu İÇİNE girip eylem seçimine karışıyor · tek başına koşumu korumaz				

14 voi-allocation Inference-Time Budget Control	Bir tavan koyduğunuzda ajan tavana kadar istediğini yapar — tavana nasıl geldiği umurunuzda değildir . Oysa aynı bütçeyle daha iyi sonuç alınabilir. Eylemleri birim bütçe başına faydaya göre puanlar. Baskı: p = 1 — min(kalan_arac/Baraç, kalan_token/Btoken) min, ortalama değil — ve fark kritik: araması bitmiş (0/3) ama token'i bol bir ajan için ortalama → p≈0,35 "rahatım" (yanlış), minimum → p=1,00 "aramam bitti" (doğru) .	Durduramaz — prompt'ta baskı notu yazar, model karar verir. Üstte deterministik guard , çünkü saf optimizasyon tehlikeli: " cevap ver " her zaman en ucuz eylemdir, yani partitka otomatik kazanır — cevabın doğruluğundan bağımsız. Ajan aceleyle kötü cevaba kaçar ve sistem bunu başarı sayar. Kanıt zayıfken erken cevap engellenir .	CEILING 300 1,050	Kör noktası tanımından geliyor: durdurmuyor. Sert tavanla birlikte kullanılmalı. Bol bütçede kazancı erir. Ölçüldü: bütçe cezası çıkarılınca F1 0,63 → 0,43 — ana sonuç "puanlama kullandık" değil, kalan bütçeyi eylem seçimine katmak .
---	--	--	--------------------------	--

03 PEKİ BU BİLGİYLE NE YAPACAĞIZ

A · Kanıt ne kadar güçlü? Aşağıdaki üç sayı başka çalışmaların ölçtükleri — sunumda alıntılatabileceğin, tartışmaya kapalı bulgular. Dördüncü kutu benim kendi ölçümüm : aynı kod, tekrarlanabilir.		B · Riski ne? — "peki zararı yok mu" sorusunun cevabı Bunlar tek tek stratejilerin değil, yaklaşımın tamamının ortak zayıflıkları. Sunumda önden söylemek, sorulmasını beklemekten iyi.		C · Koymak için ne gerekiyor? Bir sisteme guardrail eklemekten önce hazır olması gereken beş şey. Bunlar olmadan yukarıdaki mekanizmaların hiçbirisi çalışmaz.	
SAYI	NEYİN GÖSTERGESİ	RİSK	NE DEMEK	#	GEREKİLİK
%96 / 0 yanlış pozitif istatistiksel monitör: %54 / %17	Basit olan, karmaşık olanı yeniyor. Üç deterministik kural, eğitilmiş bir davranış monitöründen hem daha çok yakalıyor hem hiç yanlış alarm vermiyor . → Önce kural yaz, model eğitme.	Yanlış pozitif	Meşru bir toplu işlem "döngü" sayılıp kesilebilir. Guardrail'in kendi zararı var ve ölçülmeli.	1	İterasyon başına span. Koşum tek span'sa 4–19. adımların aynı çağrı olduğunu göremezsin. Loop detection önce bir gözlemlenebilirlik gereksinimi , sonra bir algoritma.
%45 vs %36 vs %16 4 onarım kademesi	Ajana ne söylediğin, ne yaptığından önemli. "Hangi kontrol düştü" demek, doğru cevabı vermekten iyi sonuç veriyor. → Dedektör tetiklenince dedektörün adını söyle.	Sessizce ölme	İmza yanlışsa dedektör çalışır, log basar, hiçbir şey bulmaz — ve testleri geçer , çünkü testler "yakaladı mı" diye bakar.	2	İmza normalizasyonu. Hangi alan her turda değişiyor, hangisi anlam taşıyor — envanteri çıkarılmalı. Sisteme özel; kütüphane halletmiyor. Yanlış yaparsan "sessizce ölme" riski.
\$2,45 vs \$134,50 ≈55× · aynı görev	Bütçeyi büyötmek çözüm değil. Pahalı arama yöntemi daha kötü doğruluk verdi; ayrıca 36 koşumun 21'inde yüksek reasoning effort doğruluğu artırmadı. → Daha çok harcamak yerine daha iyi durdur .	Eşikler tahmin	Dar bir limit grafikte " model kötüleşti " gibi görünür. Model aynıdır, limitin dardır — sessiz kalite gerilemesi.	3	Terminal durum ayırımı. completed / max_steps / budget / timeout / error / stuck ayrı ayrı kaydedilmeli — tek bir başarı oranı hepsini gizler.
Kendi ölçümüm — cua_lab 300 → 3 adım · 105.000 → 1.050 token · \$1,050 → \$0,011 100× ucuz 0 yanlış pozitif. 18 strateji × 3 meşru senaryo × 2 model = 108 kombinasyon ; hepsinde sütunlar none ile birebir aynı — guardrail çalışan koşuma tek token bindirmiyor . bağımlılık yok · 79 test · 16 strateji + 1 varyant		Maskeleme	Bütçe darsa dedektör hiç konuşamaz: koşum doğru durur ama teşhis kaybolur . Bütçe <i>bir şeyin</i> , döngü tespiti <i>neyin</i> yanlış gittiğini söyler.	4	Çok eksenli sayaç. Tek eksen diğerlerini gizler. Ölçüldü: betikli modelde adım ekseni bağlıyorduk, gerçek modelde süre ekseni — model hızı değişince bağlayan eksen değişti.
		Dar kapsam	Tabloda 4 satır CEILING — kör noktalar. Tek katman yetmiyor. Ve uyararak geri tepebilir (adım ekseninde ölçülmüş).	5	Nihai cevap payı. Sert limit biraz erken tetiklensin ki ajan elindekini toparlayıp sunabilsin — yoksa yapılan iş çöpe gider.
		Prompt kaynaklı döngü	Hiçbiri çözmiyor. \$01'in 5. sebebi: kaynağı kod değil, prompt.		Ayrıca Kapatma anahtarı olsun ama varsayılan AÇIK — taranan 24 harness'ın 12'sinin hatası bu. Eşikler prompt'la birlikte sürümlensin . Cl'da " tetiklenmemeli " koşumları olsun.

NEREDEN BAŞLANMALI 1 → ÖLÇ → 5

Önce **sert tavan** · sonra **izleri topla**, p99'dan **eşikleri çıkar** · sonra dedektörleri **ölçülmüş eşiklerle** yaz. *Eşiği ölçmeden dedektör yazmak, hangi sayıyı yazacağını bilmeden yazmaktır.*

Tek cümle: Model döngüyü kesemez — gereken bilgi onun görüş alanında değil. Kesme yetkisi dışarıda olmalı, ve dışarısı **ne olduğunu da söyleyebilmeli**. · sources_list/ ·

cua_lab/docs/zihniyetler.md

üç soru: kanıt ne kadar güçlü · riski ne · koymak için ne gerekiyor